ragcap: a multi-agent RAG system, built and measured

A colleague's capstone design (LangGraph router, document RAG with grade and rewrite, SQL agent, general agent, Postgres + pgvector) reproduced on local models, then evaluated the way it should be: frozen test sets, repeated runs, and a version-aware filter with an OFF/ON proof.

2019 Intel MacBook Pro · no API key · no GPU · October 2026

https://github.com/shreenathacc22/ragcap · https://shreenathacc22.github.io/ragcap/

The design to reproduce, and what was added

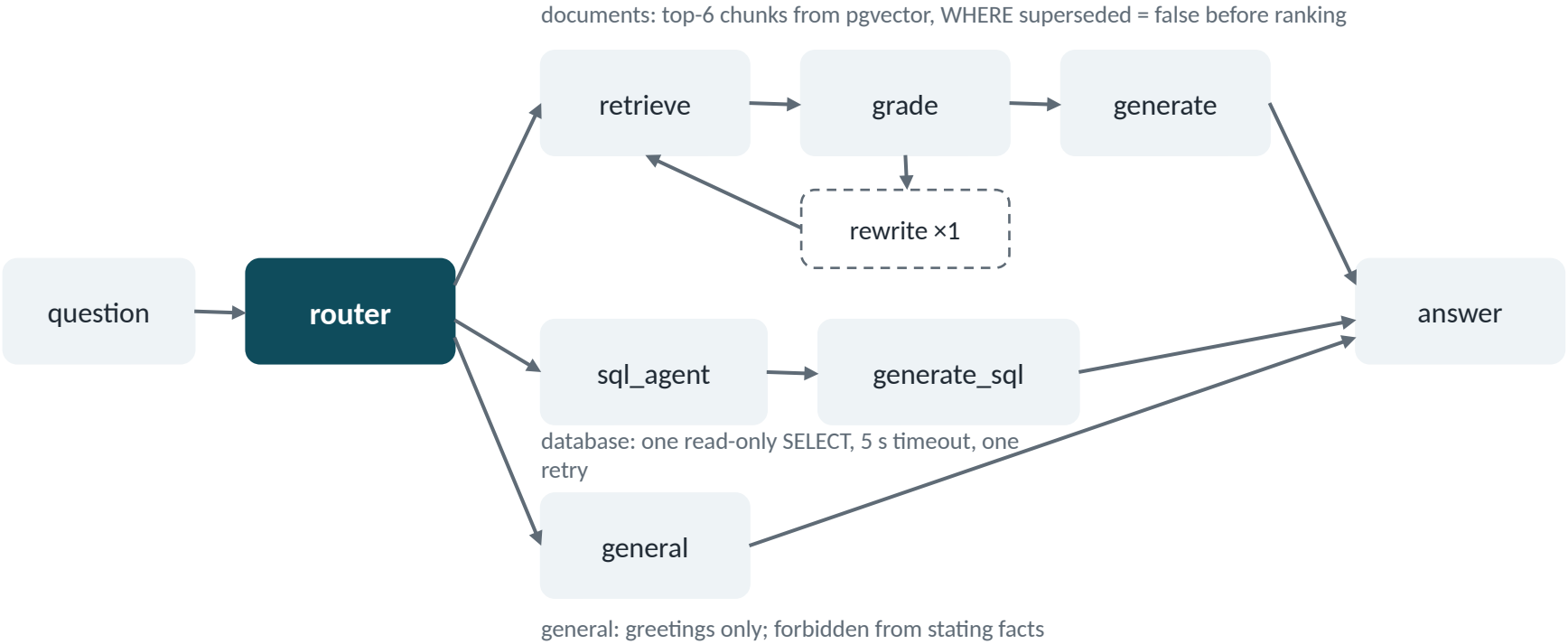
The design (from the capstone PDF)

- Router → documents / database / general
- Retrieve top-6 → Grade → one Rewrite → Generate with citations
- SQL agent: one SELECT, read-only, time-limited, one retry
- General agent never states facts
- Postgres + pgvector, LangGraph, FastAPI, streaming chat, UI
- Reported: latency and cost per flow (110 answers)

Added here

- A quality column: frozen golden set (27), unseen set (25), router / retrieval / answer accuracy
- Repeated runs: means and ranges, failing ids listed
- Version-aware retrieval: superseded versions excluded before ranking, proven OFF vs ON
- Honest negatives: 8B grader on CPU, Docling on Intel macOS
- A P&ID branch on synthetic drawings with a 3B vision model
- Three documents: SOP, runbook, recreate guide; a one-command rebuild

The graph: one router, three paths, one trace



Every node appends its milliseconds and tokens to the trace; the API streams one event per node.

The stack, all local

Postgres 16 + pgvector 0.8

In Docker on 127.0.0.1:5433. Tables documents (name, version, superseded, effective_date), chunks (vector(768), HNSW), uploaded_tables.

Ollama

llama3.2:3b for every LLM call; nomic-embed-text (768 d) for embeddings; qwen2.5vl:3b for drawings. OpenAI-compatible endpoint, so the openai SDK is the only client.

LangGraph 1.2

StateGraph with conditional edges; nodes are plain functions with a timing decorator.

Ingestion

Markdown and PDF to paragraph-packed chunks; front-matter becomes version metadata and is stripped before embedding; CSV/Excel become SQL tables.

FastAPI + web/

/upload, /chat (server-sent events), /eval, /health; a static page renders the node stream. Gradio as a second UI.

Evaluation scripts

eval.py (answer level), b1_eval.py (retrieval OFF/ON), b1_check.py (data checks + negative tests), pid_eval.py, summarize_series.py, recreate.sh.

Twenty-five steps in six phases



Build	Improve	Measure	Negative	Versions	Close out
DB, ingest, graph, golden set frozen before any code, eval, API + UI (steps 1–8)	router, SQL rules, per-passage grader; one change at a time, committed before measuring (9–14)	25 unseen questions, 3+4 repeated runs, mean and range, the gap stated (15–18)	8B grader on this CPU: recorded with evidence, switch kept for a GPU (19)	16 runbooks old/current, data checks, OFF/ON ablation, default ON (20)	browser checks, LangSmith opt-in, static front end, P&ID branch, site (21–25)

Every step in the SOP has the problem found, the change, how it was verified, and the commands used.

How the numbers were kept honest

1

Freeze first

The golden set was committed before the pipeline existed; the unseen set before its first run. Git order is the evidence.

2

Never tune on the test

A first router draft copied golden questions as examples; it was replaced before any run. Scorer fixes go in an amendments file, never in the frozen set.

3

Commit before measuring

One prompt change at a time, full run, compared by failing case ids, not just the percentage.

4

Report ranges

Three held-out runs and four tuned-set runs; the 7.6-point gap between them is stated as the over-fitting measure.

5

Fair latency

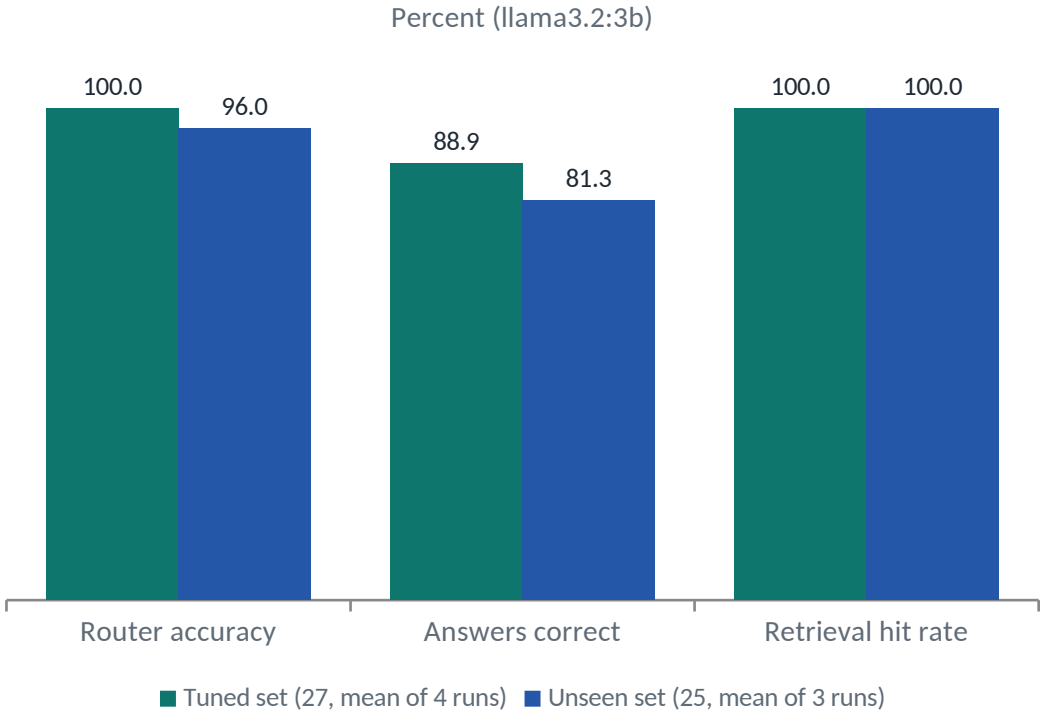
OFF and ON interleaved in one process after a warm-up; a single cold run once showed a false 3× difference.

6

Record negatives

qwen3:8b could not grade six passages in usable time on this CPU; the switch stays, the evidence is in the SOP.

Results: tuned set vs unseen set



7.6 pts

the gap between tuned and unseen answer accuracy: the honest measure of over-fitting

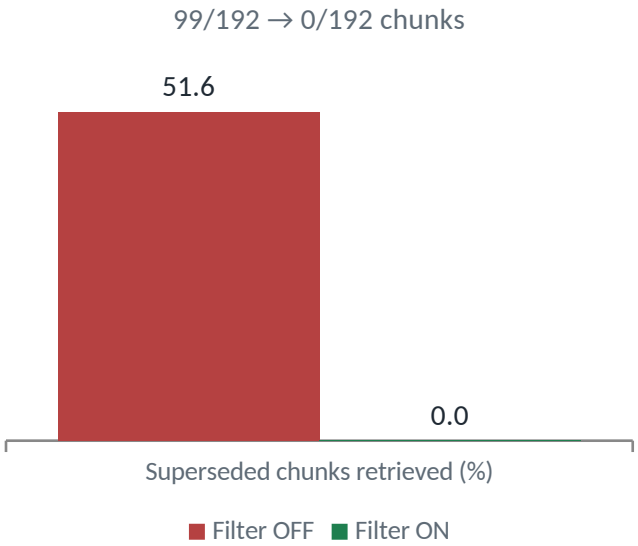
4 cases

fail in every held-out run: three are the 3B grader rejecting a differently phrased passage, one is a router ambiguity

100%

unanswerable questions refused, every run; general questions never answered with facts

Version-aware retrieval: the trap and the fix



Retrieval only, 32 frozen cases	OFF	ON
Version-trap cases passing (facts present, no stale value from an old chunk)	2/12	12/12
Control cases passing (must stay flat)	20/20	20/20
Median retrieve, interleaved ×3, 96 calls each	56 ms	57 ms
Pair similarity old vs current (nomic-embed-text)	0.962 – 0.990	

How: each document version carries version, effective_date and superseded; retrieve() adds WHERE d.superseded = false before ORDER BY, so an old runbook that is 0.99-similar to the question is never a candidate. One line of SQL, behind a switch, default ON since the ablation was recorded.

At answer level both settings scored 88.9% on this corpus (its current runbooks name their former values); multi-hop improved 0/2 → 1/2 with the filter on.

Decisions taken on evidence

Stronger grader: tested, not possible here

qwen3:8b grades one passage in 33–52 s but never returned six passages in 20 minutes on this CPU; the runner sat at 578% CPU. The escalate design and switch are in place for a GPU or hosted model.

Next.js: descoped

Node 24 is installed, so it was possible; it adds a toolchain and hosting without changing anything measured. A static page proves the streaming contract instead.

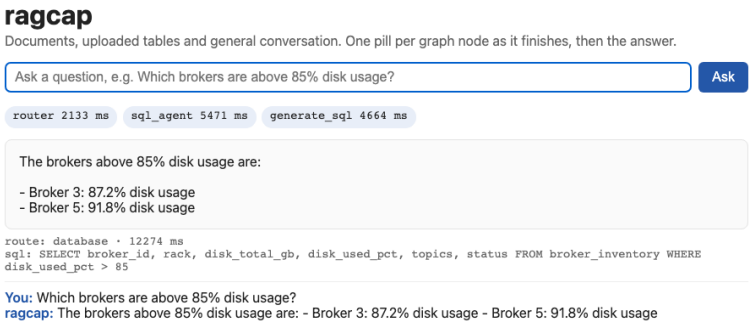
Docling: blocked on Intel macOS

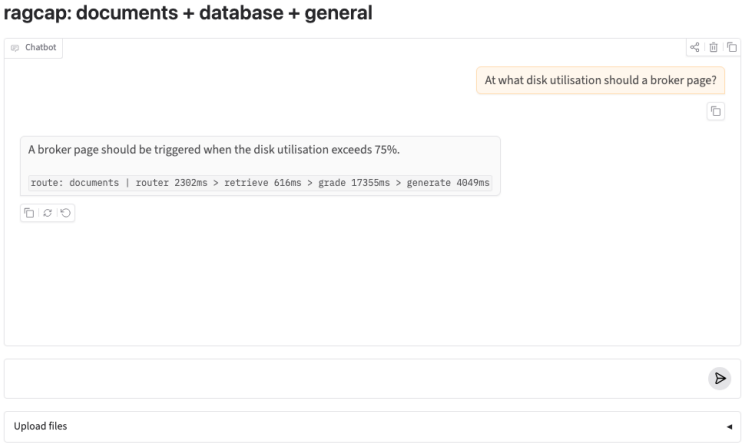
Last torch wheel is 2.2.2; the layout model needs newer transformers and numpy 2. PDFs use pypdf text; run Docling elsewhere and ingest the Markdown.

The 'outages' were sleep

Three service failures traced in the system log to the Mac sleeping; every long script now re-execs under caffeinate, and the runbook has the checklist.

Two interfaces on one streaming API

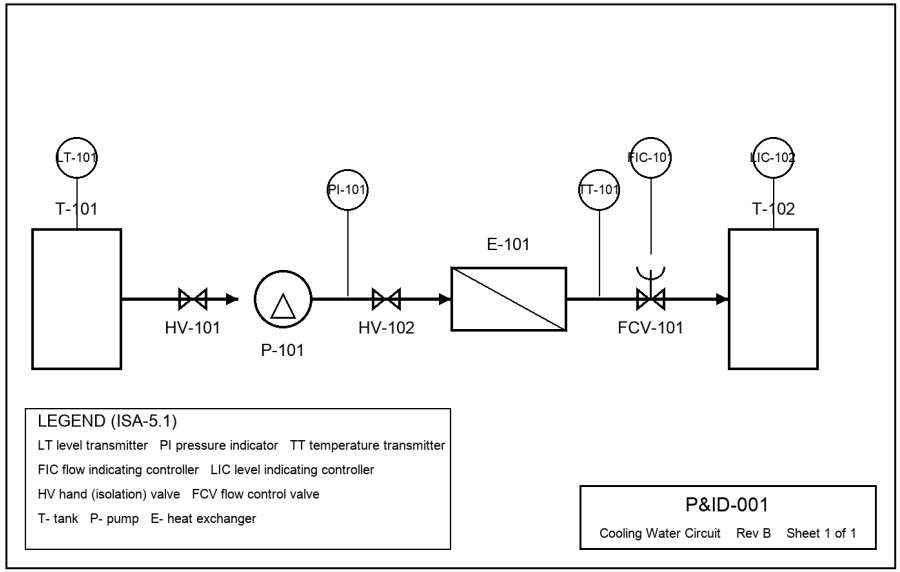




web/index.html: one pill per graph node as it finishes, then the answer, route, SQL and citations. No build step; served by FastAPI at /.

Gradio: chat with the trace line under each answer; upload documents and tables. Both were exercised in headless Chromium with live questions.

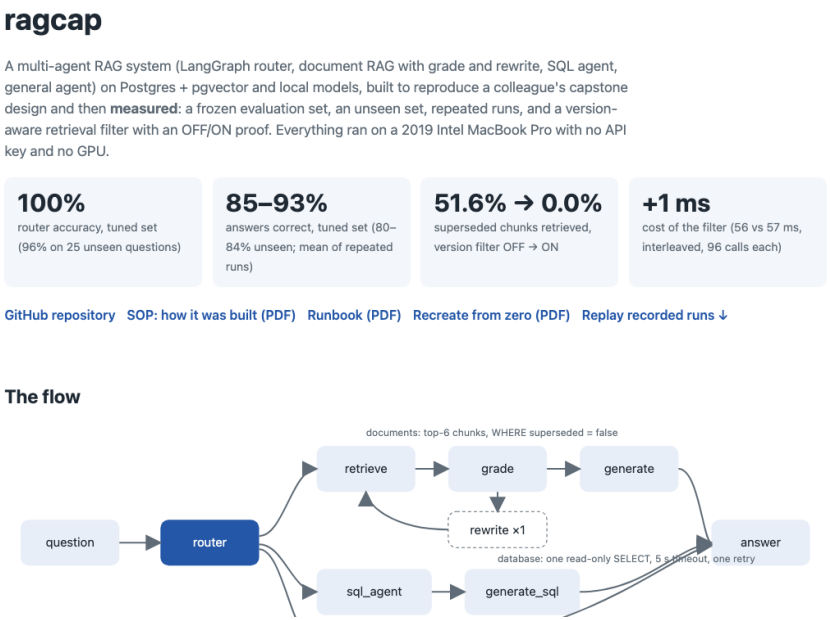
P&ID branch: drawings into the same index



Run 2 (qwen2.5vl:3b, ~100 s per drawing)	
Tags read correctly (3 drawings)	36/39, none invented
Right drawing retrieved	12/12
Answers correct, 12 frozen questions	5/12
what an instrument measures, on what	3/3
valve isolating a pump	1/3
controller + control valve	1/3
which drawing / unit	0/3

Synthetic drawings from one template with every tag in a ground-truth file (real P&IDs are confidential). What fails is the same 3B grader that limits the text pipeline, plus the title-block subtitle and small text near symbol edges: exactly what a real set will exercise first. Start a real project from a golden set like this one, on a GPU-class vision model.

See it, test it



Documents: SOP · Runbook · Recreate guide

Live showcase (GitHub Pages, static): https://shreenathacc22.github.io/ragcap/

Results, the flow, screenshots, the documents, and a replay of 64 recorded runs: pick a question, see its route, node timings, answer, and for version cases the documents retrieved OFF vs ON.

Test it live (45 min, Docker + Ollama + Python 3.12):

```
git clone https://github.com/shreenathacc22/ragcap
cd ragcap && ./recreate.sh
```

Then try the scenario catalogue in docs/RECREATE.pdf, for example:

VERSION_AWARE=0 … "At what disk utilisation should a broker page?" → often 85% (the trap)

VERSION_AWARE=1 … the same question → 75% (the fix)

"Which brokers are above 85% disk usage?" → SQL agent, brokers 3 and 5

Deploying the site to Vercel, step by step

1

Sign in

vercel.com → Continue with GitHub (the account that owns shreenathacc22/ragcap).

2

Import

Add New → Project → Import Git Repository → shreenathacc22/ragcap. Authorise the Vercel GitHub app for this repo if asked.

3

Settings

Framework Preset: Other. Root Directory: leave as the repo root. Build Command: empty. Output Directory: site (vercel.json already says so).

4

Deploy

Click Deploy; the first URL looks like ragcap-<hash>.vercel.app and opens the same site as GitHub Pages.

5

Domain

Settings → Domains → add ragcap.vercel.app (or any free name); paste the URL into README.md and the site, run publish_site.sh.

6

Keep it current

Every git push to main redeploys; the Deployments tab shows each build. CLI alternative: npm i -g vercel && vercel login && vercel --prod.

Why only the site: the pipeline needs Postgres + pgvector and Ollama, which neither Vercel nor Pages run. GitHub Pages is already live from the gh-pages branch (publish_site.sh); Vercel serves the identical site/ folder once imported.

Recreating it anywhere

```
git clone https://github.com/shreenathacc22/ragcap && cd ragcap
open -a Docker; ollama pull llama3.2:3b; ollama pull nomic-embed-text
./recreate.sh          # DB, ingest, checks, fingerprints, OFF/ON ablation, smoke questions
.venv/bin/uvicorn api:app --port 8000     # http://127.0.0.1:8000/
./run_series.sh mylabel llama3.2:3b     # repeated runs; summarize_series.py mylabel
.venv/bin/python pid_eval.py --ingest   # P&ID branch (ollama pull qwen2.5vl:3b first)
```

Fingerprint	Value a correct rebuild must match
Database after ingest	16 docs, 16 chunks, 8 superseded, 1 table
Data checks	PASS; pair similarity 0.962–0.990; three negative tests reported
Retrieval ablation	OFF 99/192 (51.6%) → ON 0/192 (0.0%); trap 2/12 → 12/12; control 20/20
Golden sets (sha1 prefixes)	golden 68ec09bd…, held-out 02ddcfd6…, P&ID 10fe1aa6…

What this shows

A design can be reproduced on a laptop with no API key. What makes it credible is not the architecture but the measurement: a frozen test set, an unseen set, ranges over repeated runs, a filter proven OFF against ON, and negatives written down with their evidence.

Next, on a GPU or hosted model: the escalate grader, a real P&ID set with its own golden questions, and LangSmith with a real key.

https://github.com/shreenathacc22/ragcap · https://shreenathacc22.github.io/ragcap/